Práctica - ABIA

Laia Beyloc, Ilyas Ouacham, Nil Muriach

Índice

Descripción....3
1. Identificación del problema....3
2. Estado del problema y representación....4
3. Representación y análisis de los operadores....6
set_operadores_1x....6
set_operadores_2....7
set_operadores_3....8
set_operadores_x....8
4. Análisis de la función heurística....9
5. Elección y generación del estado inicial....10
Estado vacío....10
Estado greedy....10
Estado aleatorio....11
Experimentación....12
1. Influencia de los operadores....12
2. Influencia de la solución inicial....14
3. Influencia de los parámetros de Simulated Annealing....16
4. Influencia del tamaño del problema en el tiempo de ejecución....18
5. Influencia del número de centros de distribución....21
6. Influencia del peso del coste por km en la satisfacción de peticiones....23
7. Influencia del horario de trabajo sobre el beneficio obtenido....25
Comparación entre Hill Climbing y Simulated Annealing....26

Descripción

1. Identificación del problema

Nos situamos en un escenario donde una empresa distribuidora de combustible debe gestionar cada día el reabastecimiento de una serie de gasolineras en una región con unos camiones cisterna.

El problema de la práctica es la asignación y gestión de recursos (que son los camiones y los viajes y kilómetros recorridos por estos) para poder hacer frente a una demanda de peticiones de las gasolineras (para llenarlas) y que la solución sea eficiente teniendo en cuenta diversas limitaciones.

En primer lugar, se debe maximizar el beneficio económico. Cada petición realizada proporciona unos ingresos (cada depósito valiendo 1000), los cuales pueden verse penalizados dependiendo de cuántos días lleven sin ser atendidos, siendo la fórmula:

$$\%precio = \left(100 - 2^{\{dias\}}\right)\%$$

En el caso de que la petición sea de ese mismo día, el beneficio asociado es del 102%.

Adicionalmente, hay que minimizar el coste de distribución, que aumenta con la distancia recorrida por los camiones (coste por km = 2). La distancia se calcula así;

$$d(D, G) = |Dx - Gx| + |Dy - Gy|)$$

siendo Dx y Dy las coordenadas del centro de distribución y Gx y Gy las de la gasolinera, pero se debe tener en cuenta que si el camión ha salido del centro, y requiere satisfacer peticiones de dos gasolineras distintas, el segundo viaje se cuenta las coordenadas del camión (que coincidiría con las coordenadas de la primera gasolinera visitada) y no de su centro de distribución, además de siempre tener en cuenta los kilómetros para volver al centro de donde ha salido.

Hay otras limitaciones de logística, como por ejemplo que cada camión solo puede hacer 640 km distancia al día (pues van a una velocidad de 80 km/h y se considera que solo van a estar haciendo una jornada de 8 horas) y 5 viajes en total. Asimismo, el camión solo tiene capacidad suficiente para llenar dos depósitos de gasolinera por viaje antes de tener que regresar (es decir, 2 peticiones por viaje). Cabe mencionar que para cada centro hay un camión disponible, así que dado que hay C centros se puede decir que hay C camiones a gestionar.

Todo esto nos lleva a concluir que es un problema de búsqueda local, ya que el espacio de todos los posibles estados es tan vasto que una exploración exhaustiva en busca del óptimo, como con A*, no parece especialmente adecuada, pues requiere de encontrar un heurístico

adecuado y admisible capaz de encontrar dicha solución antes de que se termine el espacio de memoria.

Por otro lado, la búsqueda local parte de una solución inicial y de ahí empieza a explorar con los operadores para llegar a una mejor, siempre desplazándose por el espacio de soluciones (estados que cumplen con los requisitos más no necesariamente tienen una buena calidad) con ayuda de una función heurística que estime la calidad de las soluciones candidatas y guíe la búsqueda. Por este motivo se eligen algoritmos de búsqueda local para solucionar el problema planteado en esta práctica.

En esencia, este es un problema de optimización con restricciones donde debemos buscar el espacio de posibles asignaciones que nos proporcione el mayor beneficio neto, y lo tenemos que hacer usando tanto Simulated Annealing como Hill Climbing, experimentar con cada uno y comparar sus respectivos comportamientos.

2. Estado del problema y representación

Para la representación del estado se ha intentado tener un registro mínimo de las asignaciones y valores necesarios para facilitar posteriormente el cálculo del heurístico, pues se detectó que usando una representación mínima del estado el cálculo de recorrido y el beneficio necesarios para el heurístico supondría un gran coste por cada paso del algoritmo.

La representación de estado finalmente ha sido un conjunto de atributos de clase.

Dos atributos referentes a las asignaciones:

- current_state: Una lista que representa cada gasolinera, las cuales están asociadas a la lista de gasolineras mediante los índices (es decir, el índice 2 de current_state corresponde a la gasolinera en el índice 2 con sus coordenadas). Cada índice tiene una lista que representa las peticiones. Tiene tantos índices como peticiones le corresponden y el orden representado coincide con el orden en el que se encuentran las peticiones originales. Las asignaciones se codifican dando valor “True” o “None” a cada petición según se satisfaga o no.
- camiones: Otra lista, en este caso, representa cada camión, el cual está asociado a la lista de centros de distribución mediante el índice (es decir, el índice 2 de camiones corresponde al centro de distribución en el índice 2 con sus coordenadas). Cada índice tiene una lista con un máximo de 10 valores, cada valor es una tupla que contiene en primera posición el índice de una gasolinera y en segunda el índice de una de sus peticiones o “None”. Las tuplas conceptualmente van de 2 en 2 representando los viajes que hace el camión, por lo que los valores de los índices pares no pueden tomar valor “None”, pero los índices impares sí, en caso de que el camión solo satisfaga una petición en ese viaje.

Luego tenemos atributos para facilitar el cálculo del heurístico que se actualizan al aplicar la acción:

- beneficio: Cómputo del beneficio económico aportado por las peticiones satisfechas.
- km_camiones: Una lista con los índices representando cada camión, coincidiendo con los índices de la lista camiones que guarda la cantidad de kilómetros recorridos por cada camión.
- distancia: El total de kilómetros recorridos.
- perdida_no_assignadas: El beneficio económico potencial que suponen las peticiones que no están siendo resueltas.

Asimismo, se han creado dos matrices creadas para obtener las distancias entre cada centro y gasolinera y entre cada gasolinera y gasolinera por sus índices en las listas. Puesto que en Hill Climbing se notó que se calcularía potencialmente la distancia entre unos mismos objetos múltiples veces y estas matrices evitan esa situación.

En el código que hemos hecho se puede observar en el state.py, el estado del problema se representa con la clase StateRepresentation.

Esta representación no es tan eficiente en memoria, ya que acceder a listas en Python tiene cierto coste, aunque se trabajará con los índices de las listas haciendo accesos directos la mayoría del tiempo. Pero nos facilita la aplicación de operadores y el cálculo del heurístico.

Por otra parte, el tamaño de la búsqueda total viene influida por diferentes factores combinatorios. En primer lugar, la asignación de peticiones a camiones, cada una de las P peticiones puede ser asignada a cualquiera de los C camiones, haciendo que haya C^P posibles asignaciones. El orden de atención dentro de cada camión también es un elemento de ramificación, con el camión i teniendo P_i peticiones asignadas, provocando unas $P_i!$ posibles órdenes, siendo el peor caso el reparto uniforme de estas:

$$\left(\frac{P}{C}\right)!^C$$

Así que como tamaño total aproximado, el espacio de búsqueda es:

$$C^P \cdot \left(\frac{P}{C}\right)!^C$$

Tiene un crecimiento exponencial muy grande, por lo que se observa que usar una búsqueda exhaustiva no es viable, la mejor opción es la búsqueda local con movimientos locales controlados.

3. Representación y análisis de los operadores

Al implementar los operadores se tenía en mente el objetivo de que fueran capaces de moverse por todo el espacio de soluciones, siendo capaces de realizar cambios y movimientos locales, sin salirse del espacio de soluciones y con el menor factor de ramificación posible.

Se inició trabajando con una representación de estado similar, con los mismos atributos, pero con una pequeña redundancia y más compleja y se cambió eventualmente. Antes de dicho cambio se tenían tres sets de operadores: "set_operadores_1", "set_operadores_2" y "set_operadores_3". Pero una vez se efectuó el cambio, solo se adaptó al nuevo estado "set_operadores_1", cuya adaptación es "set_operadores_1x", puesto que había demostrado mejores resultados en todos los aspectos. Así que "set_operadores_2" y "set_operadores_3" no funcionan con la representación del estado actual, pues se decidió que no valía la pena adaptarlos, pues ya habían demostrado obtener peores resultados.

Asimismo, se creó un nuevo set de operadores para el nuevo estado, basado en esos dos sets de operadores descartados, "set_operadores_x".

set_operadores_1x

Este primer set de operadores buscaba seguir una lógica lo más simple posible, basándose en un enfoque más intuitivo de lo que se puede hacer con nuestra representación de estado.

Contiene tres operadores, empezando por AddOnlyTrip, que asigna una petición individual a un nuevo viaje del camión (mientras no supere 640 km en total ni 5 viajes), actualizando el beneficio (teniendo en cuenta las pérdidas tardanza), los kilómetros y disminuyendo la pérdida potencial de las peticiones no asignadas. Dado la representación de estado, añade dos valores a la lista de camiones, el primero con la tupla conteniendo la información de la petición y el segundo con "None". Y, en la lista de gasolineras, cambia el valor de la petición de "None" a "True".

Seguido está AddMultipleTrip, el cual crea un viaje con dos peticiones distintas (si respeta los kilómetros y viajes máximos), suma el beneficio de ambas (y disminuye la pérdida potencial de las peticiones no asignadas) y actualiza la distancia recorrida por el camión (pasando por las dos gasolineras y volviendo al centro). Y actualiza las listas de asignaciones como corresponde.

Finalmente, tiene el operador RemoveTrip, que elimina un viaje existente (ya sea de una o dos peticiones), liberando las asignaciones correspondientes y reduciendo los kilómetros recorridos, pero también reduciendo el beneficio bruto que suponían las peticiones que se han liberado, además de incrementar la pérdida potencial por no atender a estas peticiones.

Respecto al coste de ramificación de cada operador. Se puede observar que AddOnlyTrip itera por cada una de las peticiones que no se han asignado (U) y los camiones disponibles (C), dando como resultado:

$$O(C \cdot U)$$

De igual manera, AddMultipleTrip tiene este mismo factor de ramificación pero con un añadido, que se combina pares de peticiones no asignadas distintas:

$$O(C \cdot U \cdot (U - 1)) \approx O(C \cdot U^2)$$

RemoveTrip tiene un coste de $O(V)$, siendo V la cantidad de viajes (parejas de tuplas o de tupla y "None") asignados en la lista camiones.

set_operadores_2

En segundo lugar, creamos otro set de operadores formado por otros tres. RemovePeticion, que elimina una petición escogida de un viaje y consecuentemente, eliminando los kilómetros recorridos por esta petición, y además, si el viaje queda vacío, se elimina, y se actualizan pérdidas y kilómetros también. En SwapPeticiones, se intercambia dos peticiones entre viajes con posiciones arbitrarias (ya sean de dos camiones o del mismo), y en el caso de que ambos no pasen el límite de kilómetros permitido, se modifican las asignaciones y distancias recorridas de los camiones. En última instancia, AddViaje inserta un viaje nuevo de una petición con un camión disponible (tiene capacidad de viajes y kilómetros suficiente), y los kilómetros correspondientes a este recado.

Para los costes de este set, empezamos con RemovePeticion, el cual funciona de la misma manera que RemoveTrip, su coste es el siguiente:

$$O(C \cdot P)$$

Donde *C* representa el número de camiones y *P* el número de peticiones

En SwapPeticiones se puede apreciar que recorre pares de peticiones entre viajes, por lo que tienes que tener en cuenta las peticiones ocupadas del primer viaje y las posiciones del segundo viaje, por lo que el coste sería todas las combinaciones posibles de dos peticiones asignadas (2 peticiones por cada viaje de cada camión como máximo):

$$O(2 \cdot C \cdot V)$$

AddViaje elige una petición no asignada (de coste $O(U)$) y mira en qué camión (C) y se puede añadir:

$$O(U \cdot C)$$

set_operadores_3

En el set de operadores 3 tenemos algunos idénticos con el 2 como SwapPeticiones o InsertViaje (igual que AddViaje). Hay otros muy parecidos como RemovePeticion, pero este en concreto registra la distancia reducida del viaje para ajustar tanto los kilómetros del camión como la distancia global recorrida y si se elimina la segunda petición en la posición 0, la de la posición 1 (si existe) se desplaza a la 0. Por otro lado, hay algunos operadores distintos, como MovePeticion, que mueve una petición de un camión a otro (con otro viaje destino por consecuencia) siempre respetando los máximos de kilómetros y peticiones por viaje, actualizando la distancia recorrida. También está FillViaje, que añade una segunda petición a un viaje ya existente (que tiene una petición) teniendo en cuenta las mismas limitaciones que el anterior y actualizando los mimos parámetros.

Para los operadores diferentes, el coste de MovePeticion para empezar tiene en cuenta todos los viajes (V) de cada camión (C) por cada petición ocupada (A), para así:

$$O(A \cdot C \cdot V)$$

FillViaje prueba de añadir a una segunda petición a viajes de un solo elemento (de una petición), mirando todas las peticiones no asignadas y viajes que admitan una segunda petición:

$$O(U \cdot (C \cdot V))$$

En este caso el coste será muy posiblemente menor, ya que solo se miran viajes que tengan una petición.

set_operadores_x

Este set contiene algunos operadores de antiguos sets, como son MovePetition y RemovePetition, y nuevos que retocan matices de otros operadores, como AssignPetition (parecido a AddOnlyTrip y FillViaje), que asigna una petición no atendida a un camión, ya sea creando un viaje nuevo o completando uno existente, teniendo en cuenta el umbral de kilómetros y viajes y actualizando el beneficio, la distancia y la pérdida potencial. Asimismo, IntraSwap es otro operador que intercambia dos peticiones dentro del mismo camión, verificando que el recuento de kilómetros es el que toca y recalculando el recorrido y su coste asociado (similar a SwapPeticiones pero más restringido).

Por lo que consta los costes asociados, el AssignPetition prueba todas las peticiones no asignadas (U) con todos los camiones C, y los viajes de 1 petición (como mucho V por camión) así que resulta en:

$$O(C \cdot U \cdot V)$$

En el IntraSwap no se consideran tantos casos como en el SwapPeticiones, ya que solo son el número de posiciones ocupadas en un camión, que está acotado por $2 \cdot V$ (2 peticiones por viaje), siendo el coste todas las combinaciones posibles de estas parejas por cada camión:

$$O\left(C \cdot (2 \cdot V)^2\right) = O\left(C \cdot V^2\right)$$

Más adelante, en el apartado de experimentación veremos cuál de los cuatro es más eficiente y es más óptima para el problema que tratamos, o una combinación de los operadores de los diferentes sets.

4. Análisis de la función heurística

Para poder realizar esta búsqueda local, es esencial tener una buena función heurística, que tenga en cuenta todos los objetivos de la búsqueda, para que funcione correctamente el algoritmo. Esto se debe a que esta parte es la que determina la "calidad" de la solución que se ha encontrado, y como más óptima sea, según se busque minimizar o maximizar, mejor calidad tendrá, y así poder guiar la búsqueda de manera estable y eficiente.

Dicho esto, para esta práctica se pide maximizar el beneficio neto y minimizar las pérdidas potenciales, así que la función heurística debería reflejar esto mismo. Es por eso que inicialmente se planteó que la heurística debería tomar una forma similar a:

$$beneficios(con\ penalizaciones) - coste\ del\ recorrido - pérdida\ no\ atendidas$$

Se puede observar que la unidad de nuestra heurística es económica. Así pues, tenemos el beneficio económico bruto conseguido por cada petición satisfecha como parte positiva a maximizar, de la función, ya que esto incentiva a seleccionar el máximo de peticiones de alto beneficio actual. Sin embargo, el principal obstáculo para el beneficio es el coste por los kilómetros que se han recorrido para satisfacer dichas peticiones, y por ello lo añadimos a la expresión de forma negativa, pues tiene el mismo peso, y está en las mismas unidades, pero se busca minimizarlo. A todo esto, añadimos también la pérdida potencial de las peticiones no atendidas, pues se pide que se minimice y supone un incentivo más para que el algoritmo valore positivamente satisfacer tantas peticiones como sea posible, pero minimizando los kilómetros que recorremos para lograrlo.

Sin embargo, se vio relevante introducir ponderaciones a cada elemento para poder balancear la heurística, porque no consideramos que todo deba tener un mismo impacto en esta. Por ejemplo, si la ponderación de los beneficios es alta comparada con la de los costes por kilómetro, se generarían rutas largas poco lógicas que estarían lejos de ser una buena solución, y si fuera el caso contrario, se favorecería prescindir de asignaciones a sitios relativamente lejanos siempre. La ponderación de las peticiones no atendidas, sin embargo, si es moderada se puede usar para reducir su cantidad, si es muy alta podría forzar viajes

marginales con una proporción desfavorable de beneficio por kilómetro, y si fuera muy baja no supondría un problema tan grande, puesto que ya se incluye motivación para satisfacer tantas peticiones como sea posible con el factor "beneficio".

Teniendo todo esto en cuenta, nos decantamos por dejar las ponderaciones de beneficios y coste del recorrido a 1, dado que son dos factores relevantes, pues no solo se deben maximizar y minimizar respectivamente, sino que son valores reales, con un peso real. En cambio, decidimos quitar peso a la pérdida de las peticiones no asignadas, pues este factor se usa para conducir el algoritmo a buscar satisfacer más peticiones, pero maximizar el beneficio ya tiene esa función y porque al no ser un valor de una pérdida económica real se consideró que debía tener menor peso. Así que consideramos que una solución que maximice esta heurística será una solución con una gran calidad.

La función heurística finalmente es:

$$beneficios(con\ penalizaciones) - coste\ del\ recorrido - 0.5 \cdot pérdida\ no\ atendidas$$

5. Elección y generación del estado inicial

En este proyecto, la estrategia de generación del estado inicial (estado que se genera al inicio y del que parte la búsqueda) se ha implementado de tres maneras diferentes: el estado vacío, el estado "greedy" y el estado aleatorio.

Estado vacío

Primeramente, en el estado vacío se inicializa current_state con todas las peticiones a None (ninguna está asignada), las listas con las rutas de los camiones vacías y el recuento de kilómetros a cero. Al no haber ninguno de estos elementos, el beneficio total inicial empieza en 0, y la pérdida por no asignar peticiones será máxima. Este estado inicia con una heurística negativa.

De las tres opciones, este estado es con diferencia el más "barato" de construir tanto en referencia a tiempo y memoria, así como el más simple.

En cuanto a la bondad esperada para este tipo de generación, tiene una calidad baja, por no decir mínima. Ahora bien, puede ser muy intuitivo para pensar en la aplicación de operadores y planteamiento del problema. Potencialmente, este estado inicial requerirá de más pasos para converger, puesto que está muy alejado de un heurístico aceptable.

Estado greedy

El estado greedy se construye agrupando gasolineras al centro más cercano, después se ordenan peticiones por beneficio decreciente agrupadas por proximidad geográfica (con

clústeres), se añaden viajes de una o dos paradas haciendo (si es posible) que si hay una segunda petición sea la gasolinera más cercana a la primera. Seguidamente, se valida cada ruta por su valor de distancia recorrida mientras se respeta los límites de kilómetros y viajes.

El coste de creación de esta opción es moderado, provocado por el orden y las comprobaciones de ruta que se ejecutan, que hace que sea bastante más elevado comparado con el anterior. En el inicio el beneficio acostumbra a ser bastante alto (dado a estas prioridades al crear el estado), el coste por el recorrido razonable y la pérdida que falta proviene de las peticiones no asignadas, que tampoco debería ser muy elevada.

Por lo que respecta a la bondad esperada, el estado greedy se caracteriza por tener una buena proporción de beneficio por distancia, ya que explota la cercanía e intenta combinar dos peticiones por viaje, y así acelera el proceso de la búsqueda local y deja menos ediciones de estado para mejorar la solución.

Estado aleatorio

Para terminar, el estado aleatorio tiene como base partir de rutas vacías, en las cuales se asignan peticiones libres aleatoriamente, y también se asignan camiones de esta manera siempre y cuando estén dentro del intervalo máximo de kilómetros y viajes por camión.

A diferencia del greedy, el coste suele ser bajo, aunque puede ser moderado dependiendo de los intentos aceptados para crear un estado que respeta las precondiciones. Al ser aleatorio, no se puede estimar el valor que tendrá de beneficio y pérdidas del estado, pero sí que se sabe que el beneficio del día irá incrementando con cada asignación y reducirá la perdida por no asignación, al mismo tiempo que crecerá la distancia recorrida gracias a las rutas que se hagan.

La bondad esperada puede variar mucho dependiendo de como salgan los valores del estado,

y tampoco ordena por beneficio, aunque es útil para una diversidad inicial en comparación con el empty y sacrifica calidad respecto al greedy a cambio de un menor coste de construcción.

Usaremos estos estados en diferentes casos durante la experimentación, para ver cuál es más eficiente y con qué operadores funciona mejor para darnos la mejor heurística.

Experimentación

Para hacer estos experimentos se asume que los datos obtenidos siguen una distribución normal. Usaremos gráficas para observar las diferentes tendencias de los valores a analizar y tests estadísticos para confirmar o no nuestras hipótesis.

1. Influencia de los operadores

Para determinar la influencia de los operadores en el proceso de búsqueda de la solución optimizada se ha realizado el experimento 1.

Con este experimento se observará si hay una diferencia estadística entre los resultados y cómo se desenvuelven los sets de operadores "set_operadores_1x" y "set_operadores_x".

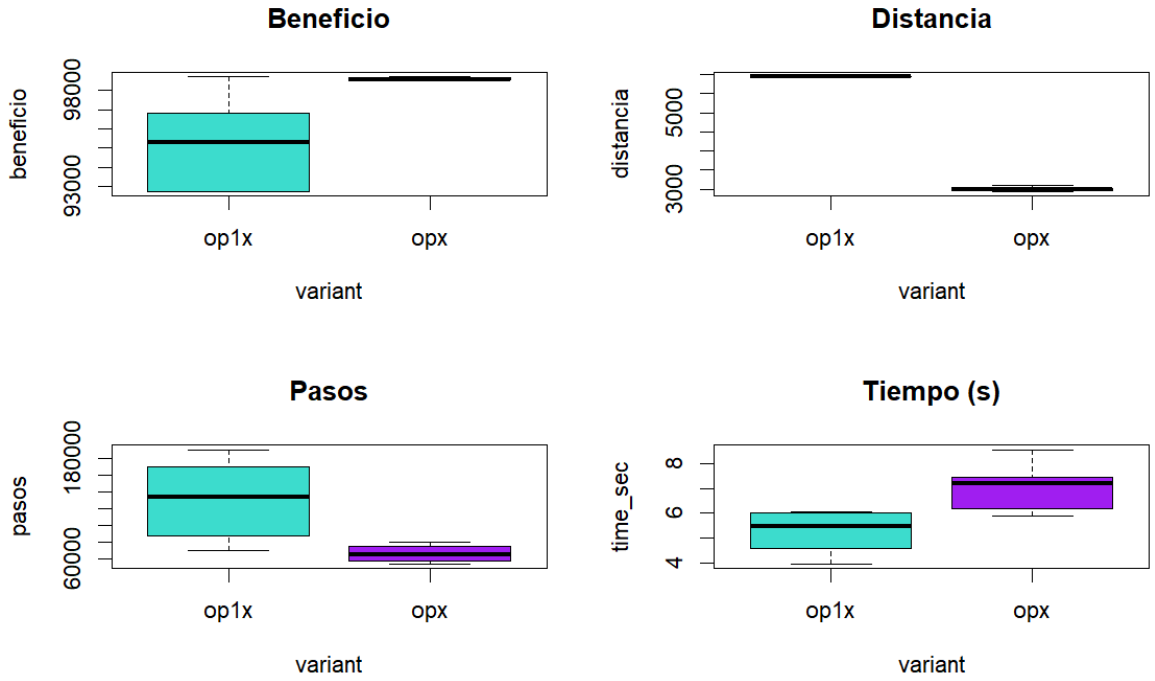
El objetivo ha sido seleccionar el set de operadores que finalmente será usado para resolver el problema. Se determinará iniciando con un mismo generador de la solución inicial y el algoritmo Hill Climbing.

Para ello se van a realizar unos T-Tests para observar si la diferencia entre sus medias obtenidas para el beneficio no neto, la distancia recorrida, la cantidad de "pasos" ejecutados y, finalmente, el tiempo que requieren para ejecutar cada repetición son estadísticamente equivalentes.

En caso de no serlo, se observarán los boxplots comparando los valores para estas variables, determinando finalmente qué set de operadores es superior.

Para este experimento se espera obtener que el set de operadores “set_operadores_x” superaba a “set_operadores_1x” en número de pasos y tiempo de ejecución, aunque llegando a un beneficio y distancia recorrida equivalente. Esto se debe a que “set_operadores_x” incluye operadores más expresivos como “MovePetition” e “IntraSwap”, que proporciona herramientas para ajustar mejor las rutas y reubicar peticiones entre viajes del mismo camión sin necesidad de crear o eliminar viajes adicionales ni modificar varias variables . En concreto, “IntraSwap” puede reducir kilómetros dentro de un mismo camión sin alterar las peticiones atendidas, mientras que “MovePetition” permite resolver estancamientos, trasladando peticiones a viajes donde se permita, algo que “set_operadores_1x” no puede hacer de forma directa. “set_operadores_1x” puede llegar a todas las soluciones posibles, pero no puede moverse tan libremente por el conjunto de soluciones teniendo operadores que modifiquen el heurístico sin cambiar las peticiones que se satisfacen y las que no, como hace el set de operadores “set_operadores_1x”.

El estado inicial del experimento ha sido, para los dos sets de operadores, el estado "Greedy", que con la seed aleatoria tenía un beneficio bruto inicial de 85.000.



Los T-Tests efectuados en el archivo “ExperimentacionEstadistica.rmd” indican que hay una diferencia estadísticamente significativa para todos los valores observados.

Puesto que el set de operadores “set_operadores_x” obtiene valores estadísticamente superiores, será el elegido para el proyecto.

Este set maximiza la heurística, minimizando la distancia recorrida y maximizando el beneficio, de forma más estable, dado que exhibe menor variabilidad.

También necesita un número significativamente menor de pasos para converger, pues sus operadores permiten una mayor flexibilidad para desplazarse por el espacio de soluciones de forma rápida y efectiva. Sin embargo, esto tiene su desventaja.

Las operaciones realizadas por sus operadores de “MovePetition” e “IntraSwap” solo son replicables con el set de operadores “set_operadores_1x” usando 2 y 4 pasos respectivamente, por ello observamos una mejora importante en cuánto a cantidad de pasos requerida.

Sin embargo, esto supone también un mayor factor de ramificación para el set “set_operadores_x” implicando un mayor tiempo de ejecución, observable en el cuarto boxplot.

Puesto que en este sentido el set de operadores “set_operadores_x” no es claramente superior, basamos nuestra elección en el argumento que el set hace un trabajo significativamente mejor optimizando la heurística.

2. Influencia de la solución inicial

Para determinar la influencia de la solución inicial en el proceso de búsqueda de la solución optimizada se ha realizado el experimento 2.

Con este segundo experimento se observa si hay una diferencia estadística entre los resultados obtenidos partiendo de la solución inicial "empty", "random" y "greedy".

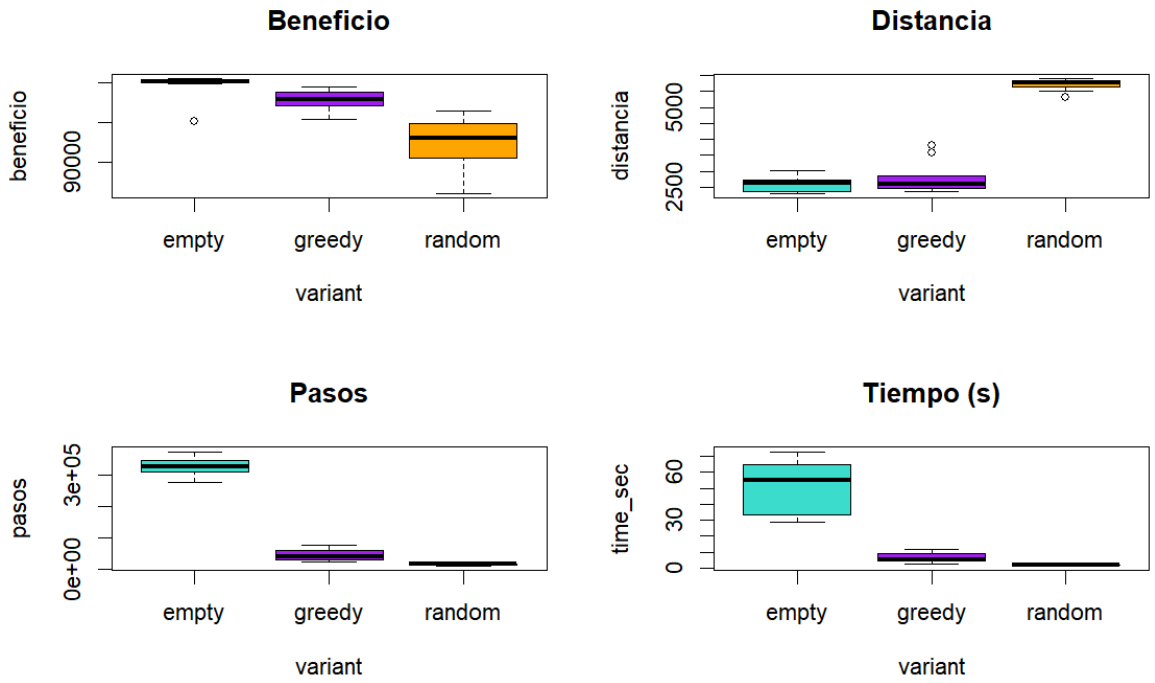
El objetivo ha sido elegir el generador de estado inicial que finalmente se usará para resolver el problema. Para experimentar se ha usado el set de operadores "set_operadores_x" y el algoritmo Hill Climbing.

Para lograr el objetivo se van a realizar, nuevamente, unos T-Tests para observar si la diferencia entre las medias obtenidas para el beneficio no neto, la distancia recorrida, la cantidad de "pasos" ejecutados y, finalmente, el tiempo que tardan en ejecutar cada repetición son estadísticamente equivalentes.

En caso de no serlo, se observarán los boxplots comparando los valores para estas variables, determinando finalmente qué solución inicial proporciona los mejores resultados según nuestro criterio.

En términos generales, se espera que el estado inicial “greedy” supere a “random” y “empty” en beneficio neto final y distancia total recorrida. Puesto que parte de rutas con una buena proporción de beneficio por kilómetro y buen valor del heurístico, lo que facilita a la búsqueda con Hill Climbing arreglar en pocos pasos la solución, pues ya tiene una calidad moderada.

Para obtener resultados más variados, se ha decidido iniciar cambiando cada vez los parámetros iniciales.



Los T-Tests realizados en el archivo ExperimentacionEstadistica.rmd indican que existe una diferencia estadísticamente significativa entre los tres estados iniciales para todas las variables observadas, excepto en la distancia recorrida entre “empty” y “greedy”.

La estrategia de generación de la solución inicial seleccionada finalmente ha sido “greedy”.

Aunque el estado inicial “empty” obtiene un rendimiento estadísticamente superior al de “greedy” en la variable beneficio (p-value = 0.008148), presenta un comportamiento claramente inferior en cuanto al número de pasos y al tiempo de ejecución, con p-values de 5.971e-14 y 5.87e-06 respectivamente.

A pesar de que “greedy” no es la estrategia inicial que maximiza más el beneficio, sus valores se encuentran en un rango similar al de “empty” (beneficios medios de 100230 para “empty” y 97850 para “greedy”). Sin embargo, en lo referente al número de pasos y al tiempo de ejecución, “greedy” obtiene valores significativamente más bajos, mostrando una superioridad clara respecto a “empty”.

En contraste, la estrategia “random” muestra un beneficio medio inferior (92980) y una distancia recorrida considerablemente mayor (5756). Esto indica que “random” posee una menor capacidad para guiar el proceso de búsqueda hacia soluciones heurísticamente favorables, quedando por detrás de “empty” y de “greedy” a pesar de conseguir converger más rápidamente; por ello no ha sido seleccionado.

3. Influencia de los parámetros de Simulated Annealing

Para determinar los parámetros que dan mejores resultados para el algoritmo de Simulated Annealing hemos empezado haciendo un estudio del progreso del beneficio y las pérdidas respecto a los pasos tomados, para cada set de parámetros generados con el producto cartesiano entre valores de K y valores de Lambda (el límite de iteraciones para comenzar estaba en 100000, para todas las combinaciones).

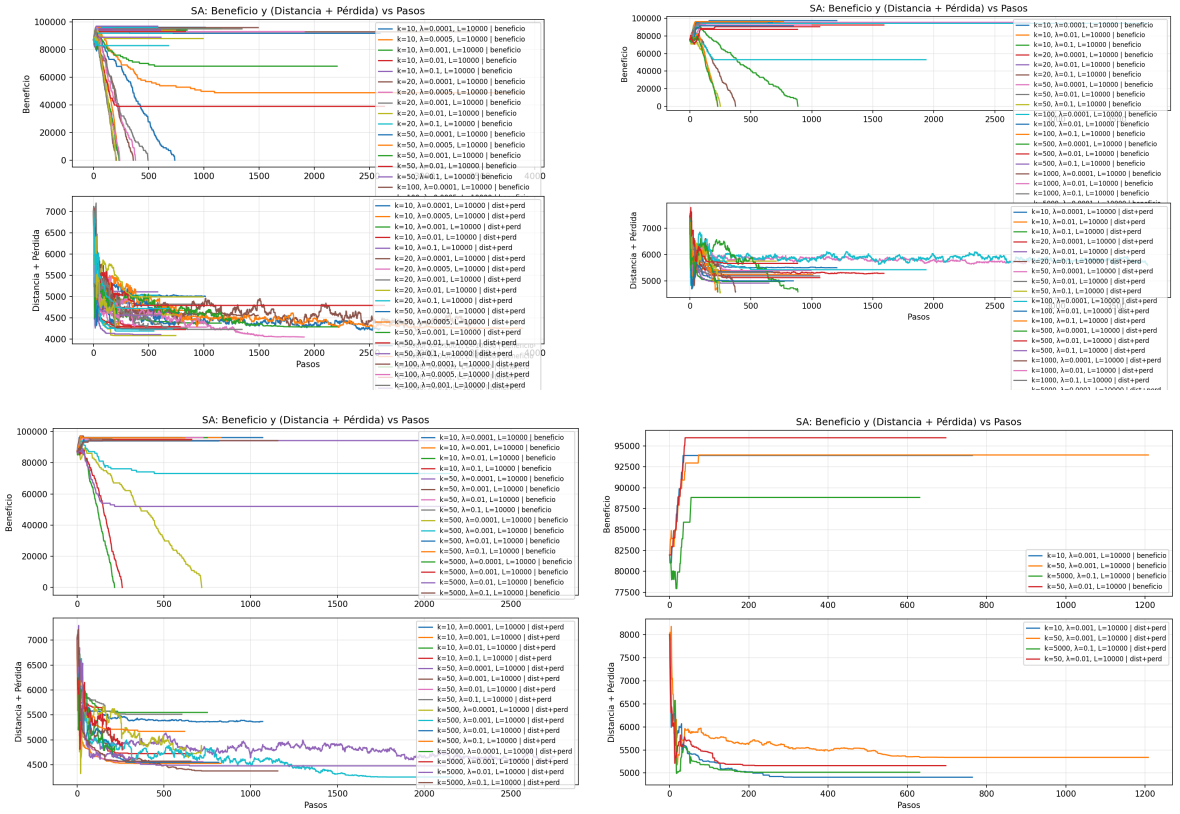


Figura 1: gráficas comparando diferentes parámetros para la función *schedule* de *Simulated Annealing*

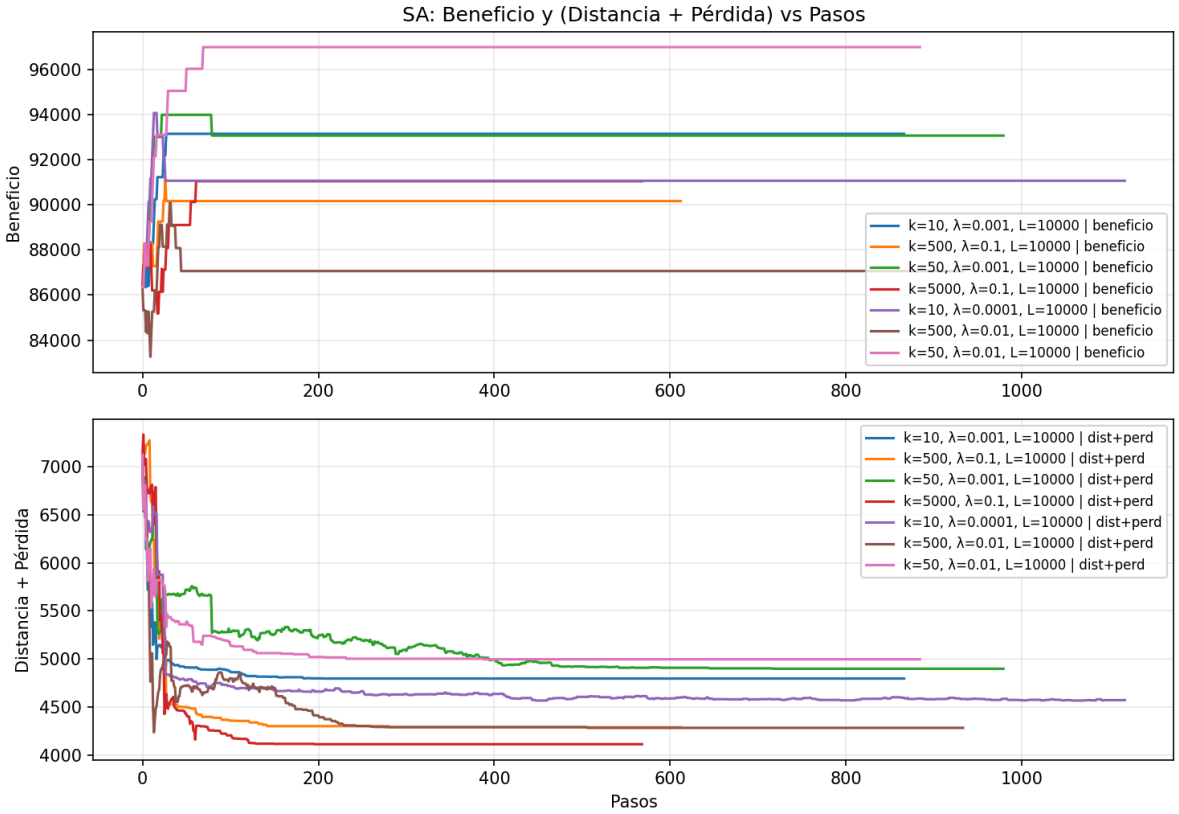


Figura 2: gráficas comparando parámetros de SA con la variación de beneficio y pérdidas por paso

Con los parámetros dados, tenemos una idea de por donde nos hemos de mover. Además, cabe destacar que al partir desde el estado generado de forma "greedy", ya tenemos una solución de calidad razonable, por lo que los valores de temperatura y enfriamiento (como podemos observar en las figuras 1 y 2) deberán tender a lo bajo y a lo alto, respectivamente. No obstante, no podemos estar seguros de que los valores de enfriamiento deban ser altos.

Así pues, para Lambda = 0.01, a continuación tenemos los valores del beneficio neto medio (beneficio - distancia recorrida x 2) según el valor de la K:

K	5	10	20	50	500	1000
Beneficio neto	89712	88482	89464	89506	88212	79729

Figura 3: tabla comparativa de beneficio neto respecto el valor de K

De esta tabla seleccionamos los 3 mejores resultados y pasamos a evaluar (para la mejor K por ahora) los valores de Lambda.

Lambda	0.5	0.1	0.01	0.001	0.0005	0.0001
Beneficio neto	89134	87698	89478	89777	90858	89529

Figura 4: tabla comparativa de beneficio neto respecto el valor de Lambda

	Lambda = 0.001	Lambda = 0.0005	Lambda = 0.0001
K = 5	88658.0	89121.0	88746.8
K = 20	89086.4	90102.0	90079.0
K = 50	89578.4	90178.2	87931.0

Figura 4: tabla comparativa de beneficio neto respecto el valor de K y Lambda

Como podemos apreciar, los valores ganadores son K=50 y Lambda=0.0005. Cabe destacar que los valores con el beneficio máximo (no el medio) son K = 5 y Lambda = 0.001, con 95212 de beneficio neto. Aunque también se introduce mucha variabilidad, siendo el mínimo un valor de beneficio neto de 86420. Por lo tanto, nos quedamos con K = 50 y Lambda = 0.0005.

4. Influencia del tamaño del problema en el tiempo de ejecución

Para determinar la influencia del tamaño del problema en el tiempo de ejecución se ha realizado el experimento 4.

Con este experimento se observa cómo evoluciona el tiempo de ejecución tomado para hallar la solución según se aumenta el valor de los parámetros siguiendo la proporción 10:100 para centros y gasolineras.

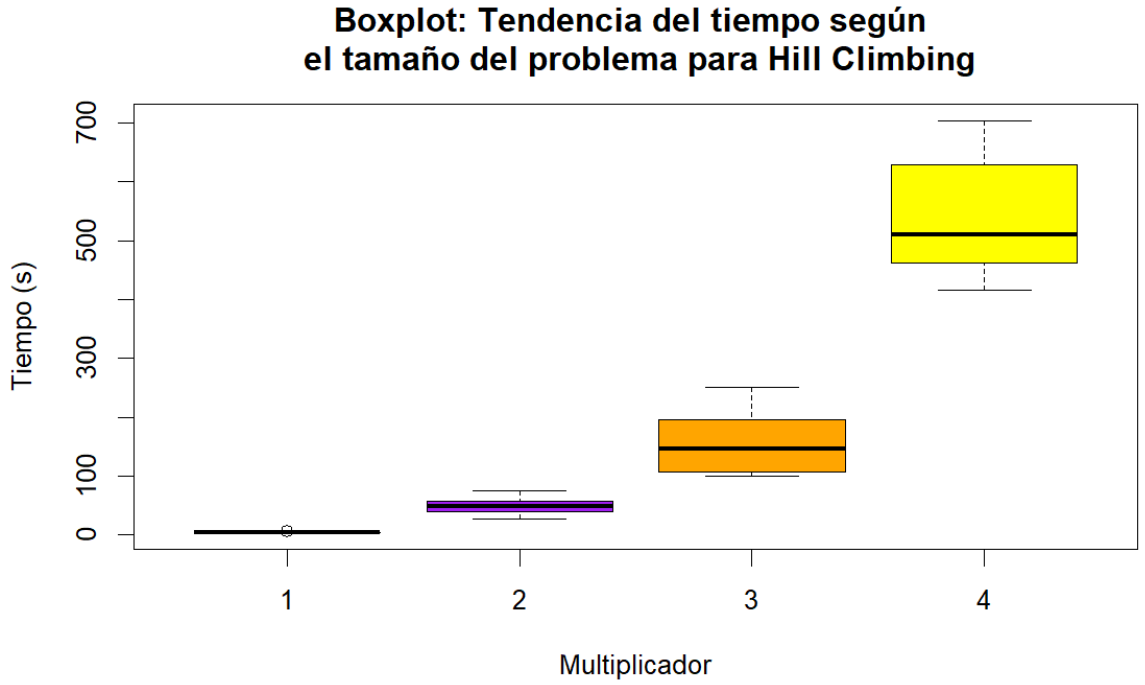
Se inicia con 10 centros de distribución y se incrementa el número de 10 en 10 hasta 100, entonces se observa la tendencia; si no se identifica se repetirá el experimento hasta añadiendo 50 centros más de 10 en 10.

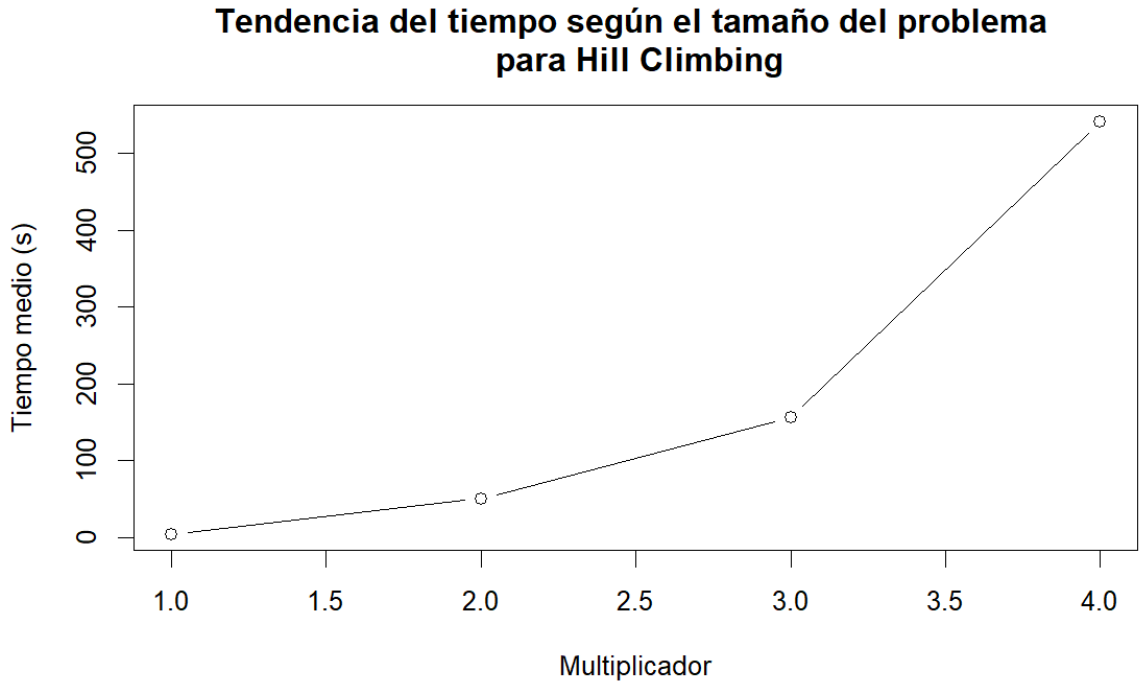
Para aumentar la cantidad de centros de distribución se ha añadido un multiplicador (no confundir con la multiplicidad).

Se usarán los algoritmos de Hill Climbing y Simulated Annealing y la misma función heurística que antes; también se iniciará desde el estado inicial “greedy” y se usará el set de operadores “set_operadores_x”.

Se comprobará que los parámetros para Simulated Annealing hallados en el apartado anterior siguen dando buenos resultados al aumentar el tamaño del problema.

En primer lugar, se ha estudiado la tendencia del tiempo de ejecución para Hill Climbing.



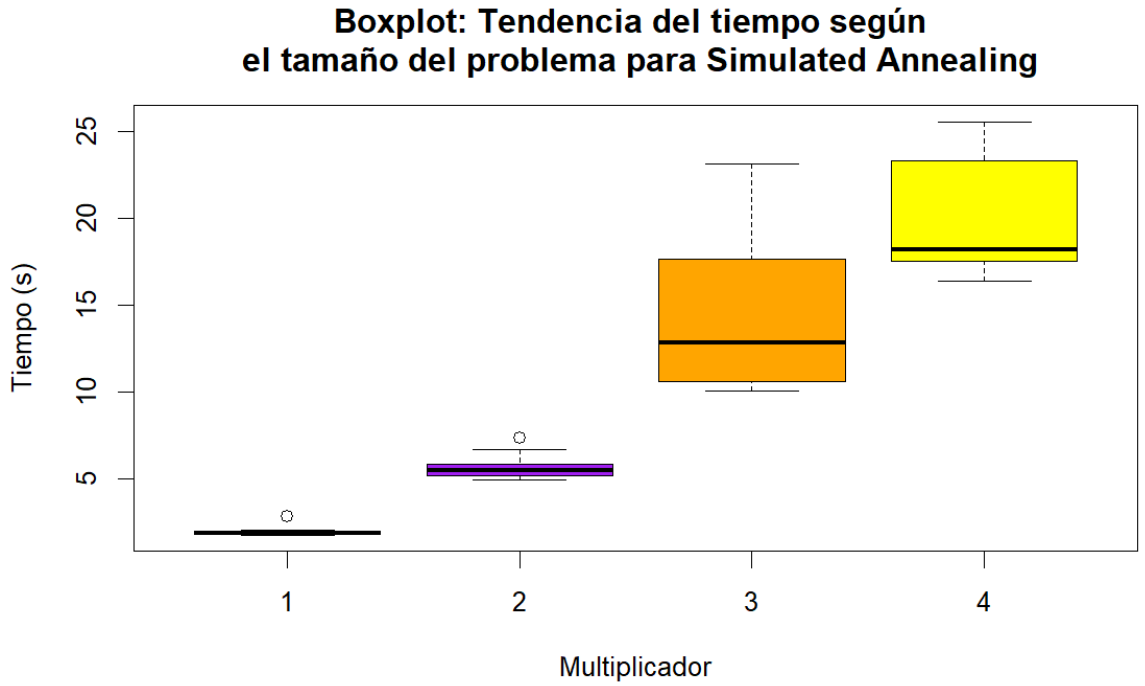


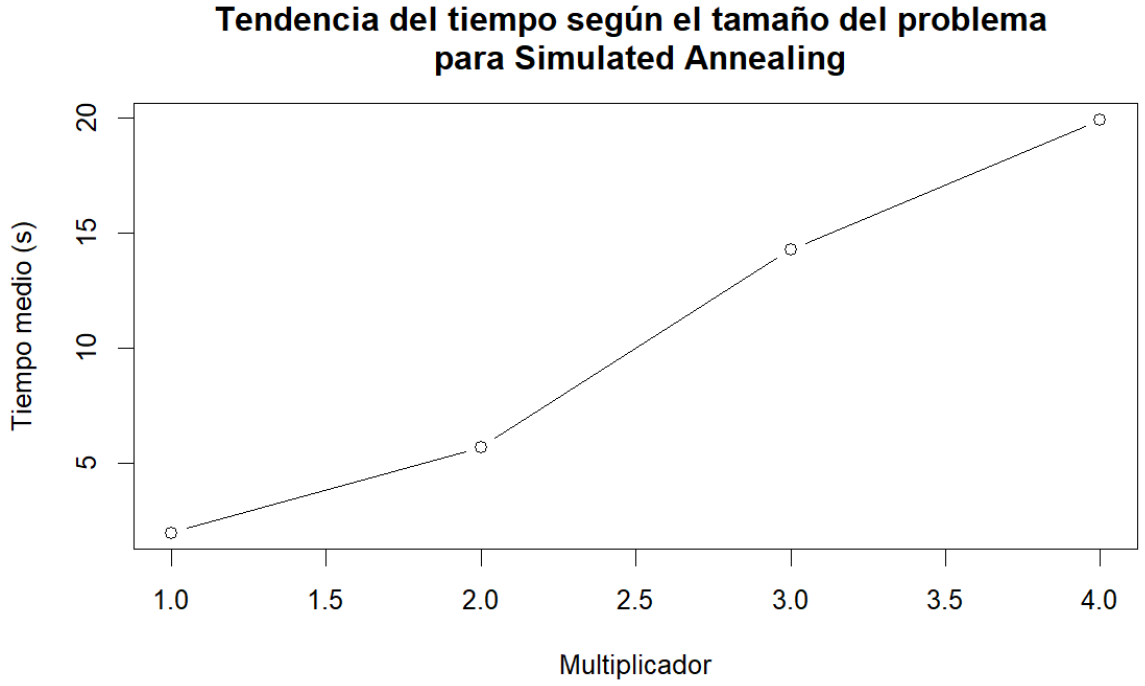
La primera gráfica muestra claramente que cada incremento en el tamaño del problema produce un aumento pronunciado del tiempo de ejecución para cada repetición, pasando desde valores muy bajos en el multiplicador 1 hasta tiempos superiores a los 500 segundos cuando el multiplicador alcanza 4.

La segunda gráfica confirma esta observación, pues el tiempo medio crece de forma casi exponencial conforme aumenta el multiplicador, especialmente a partir del multiplicador 3, donde el incremento se vuelve mucho más acusado. Del mismo modo, se aprecia que la varianza del tiempo de ejecución también aumenta a medida que crece el tamaño del problema.

En resumen, el experimento muestra que el tiempo de ejecución de Hill Climbing escala de manera claramente no lineal, posiblemente exponencial, respecto al tamaño del problema, y que la dispersión de tiempos también crece significativamente, posiblemente como $Var=\mu^2$.

Seguidamente, se ha estudiado la tendencia del tiempo de ejecución para Simulated Annealing.





En la primera gráfica se puede observar claramente que cada incremento en el tamaño del problema produce un aumento lineal en el tiempo de ejecución del programa, así como un aumento exponencial en su variabilidad.

Comparativamente con Hill Climbing se puede observar que el crecimiento no es tan pronunciado, con multiplicidad 4 uno tarda alrededor de 500 segundos para ejecutarse y el otro únicamente 20 segundos. Mostrando la clara ventaja de Simulated Annealing por su factor de ramificación de O(1).

Sin embargo, el Simulated Annealing depende de unos buenos parámetros para asegurar llegar a una buena solución final, pues aunque no sea tan sencillo que acabe estancándose como el Hill Climbing, puede aún así no lograr mejores resultados.

5. Influencia del número de centros de distribución

Para determinar la influencia del tamaño del número de centros de distribución se ha realizado el experimento 5.

Con este experimento se busca observar cómo afecta reducir el número de centros de distribución a la mitad; aunque manteniendo la cantidad de camiones cisterna.

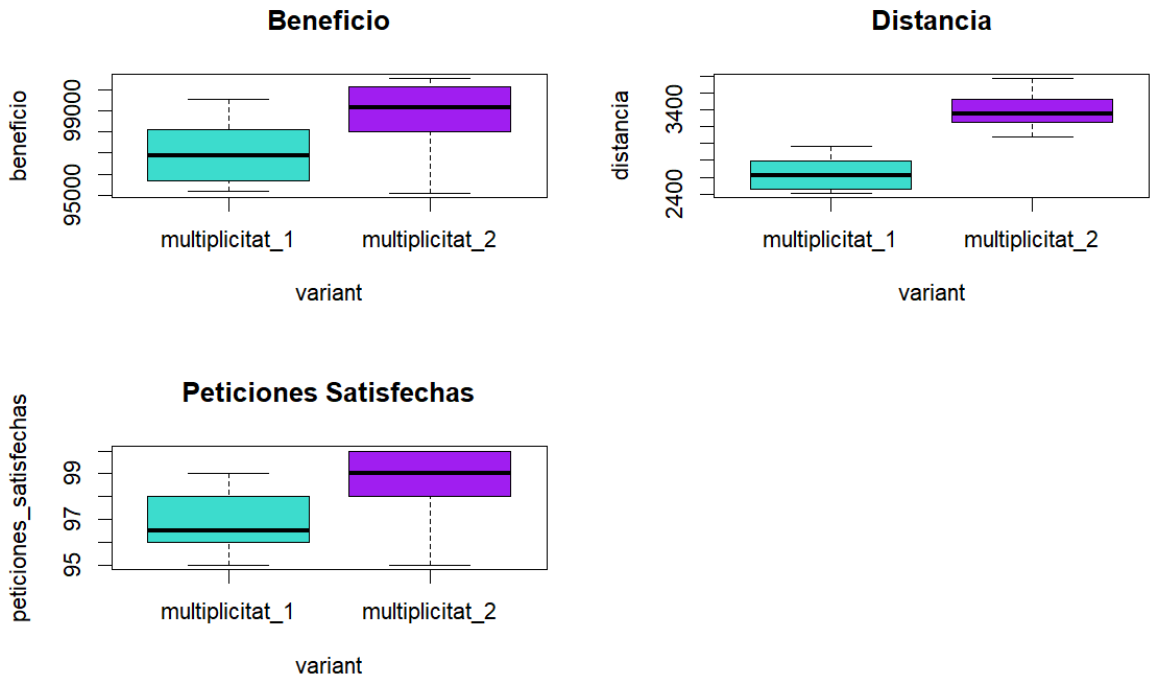
Para ello se ha decidido aumentar por dos la multiplicidad (que originalmente era 1), y reducir el número de centros de 10 a 5, manteniendo el número de gasolineras como 100.

El objetivo es analizar cómo afecta este cambio al beneficio y a los kilómetros recorridos. Y si se sigue sirviendo el mismo número de peticiones o no.

Se hará dicho análisis a partir de la comparación estadística de los valores obtenidos con cada modelo, el original y el planteado en este apartado.

Al reducir de 10 a 5 los centros de distribución, pero mantener el mismo número de camiones (incrementado la multiplicidad a 2), la predicción es de un aumento a la distancia media por viaje debido a mayores desplazamientos desde cetros lejanos de parte de las gasolineras. En consecuencia, el coste por kilómetro debería subir y el beneficio neto debería bajar comparado con el primer escenario.

El experimento se ha realizado desde el estado inicial "greedy", usando Hill Climbing como algoritmo y los operadores "set_operadores_x" para desplazarse por el espacio de soluciones.



Los T-Tests realizados en el archivo "ExperimentacionEstadistica.rmd" indican que existe una diferencia estadísticamente significativa entre los dos modelos para la distancia recorrida y la cantidad de peticiones satisfechas; sin embargo, no hay una diferencia estadística significativa para el beneficio obtenido.

Reducir el número de centros de distribución, aun manteniendo el número de camiones cisterna, hace que el modelo obtenga un mismo beneficio, aun satisfaciendo más peticiones, con un mayor coste, pues cubre más distancia.

6. Influencia del peso del coste por km en la satisfacción de peticiones

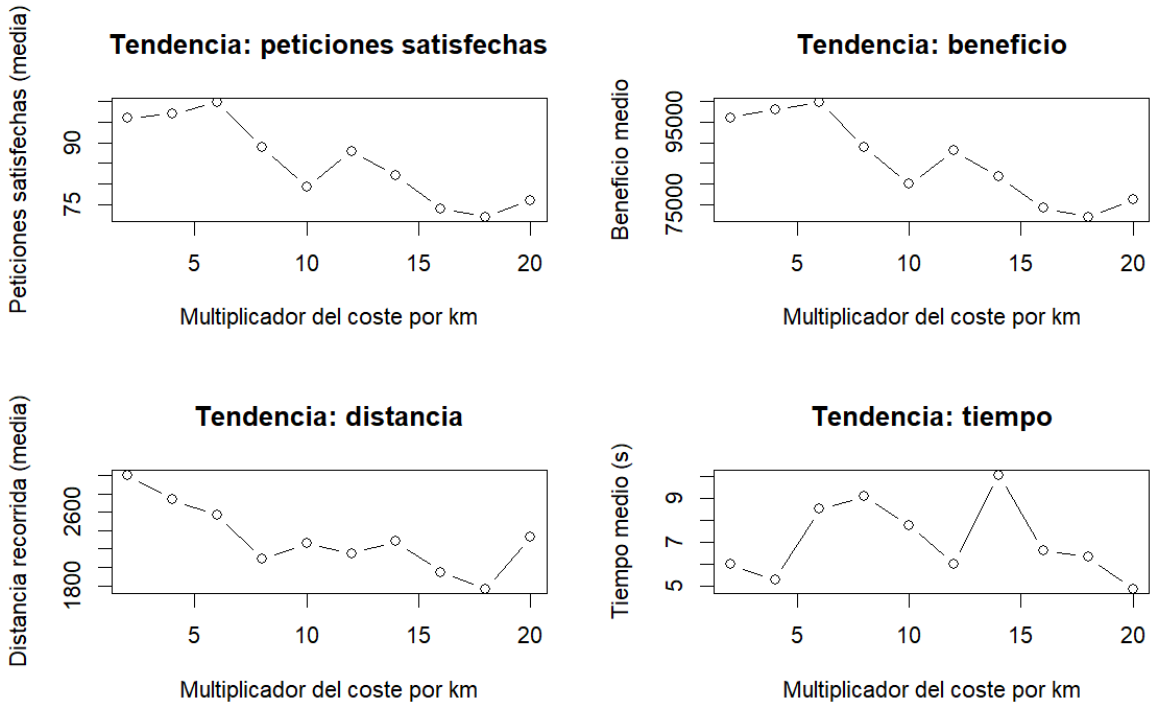
Para determinar la influencia del peso del coste por km en la satisfacción de peticiones se ha realizado el experimento 6.

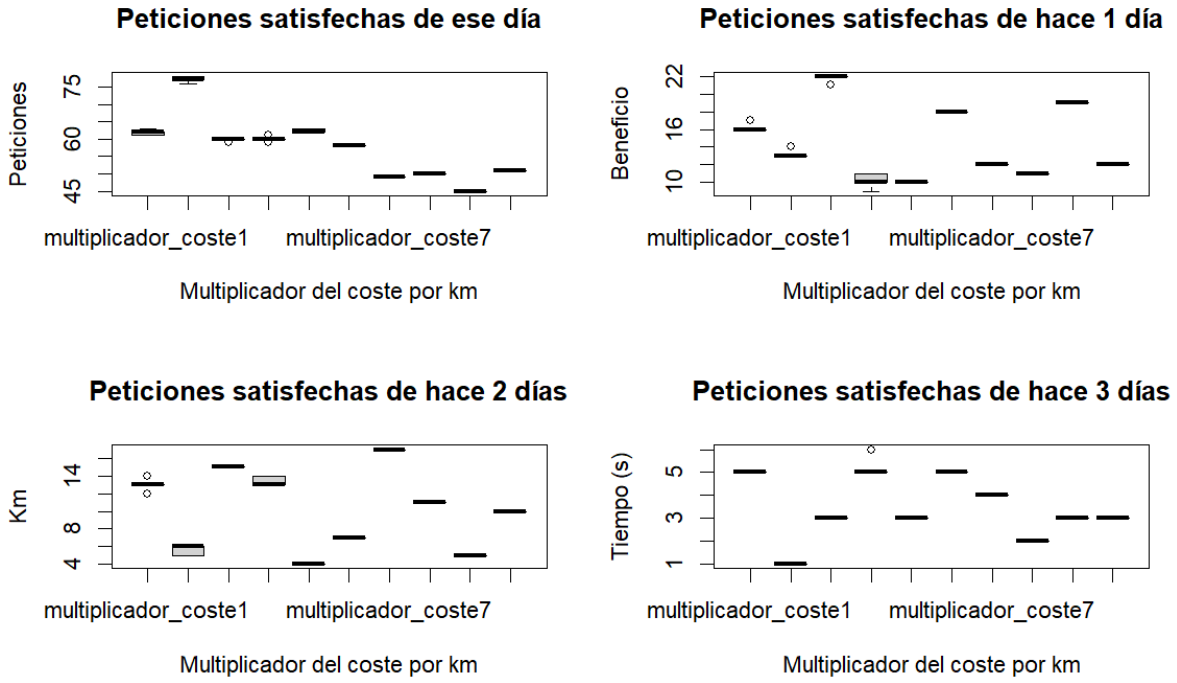
Con este experimento se busca determinar cómo afecta doblar el coste de la distancia a las peticiones que se satisfacen y qué tendencia se observa respecto al beneficio y distancia recorrida, así como cómo varía la proporción de días pendientes de las peticiones satisfechas.

Para ello hemos realizado 10 experimentos desde el coste 2 originalmente planteado hasta coste 20.

El experimento se ha iniciado usando la estrategia de generación del estado inicial “greedy” y se ha usado Hill Climbing como algoritmo, así como el set de operadores “set_operadores_x”.

Como expectativa del resultado de este experimento, al doblar progresivamente el coste por kilómetro desde 2 hasta 20, se esperaba una disminución clara del beneficio neto y una reducción de la distancia total recorrida, ya que Hill Climbing tenderá a favorecer soluciones con menos kilómetros y con viajes más cortos (y unitarios) para poder maximizar el beneficio.





Las primeras cuatro gráficas muestran las variables principales, peticiones satisfechas, beneficio, distancia recorrida y tiempo.

A medida que aumenta el multiplicador del coste por km, disminuye de forma clara y consistente el número de peticiones satisfechas. Esta caída es especialmente pronunciada a partir de multiplicadores superiores a 10 (nótese que hemos mirado el valor de 10 multiplicadores que van de 2 a 20 de 2 en 2).

El beneficio sigue una tendencia similar a la de peticiones satisfechas. Aunque los valores presentan pequeñas oscilaciones, el comportamiento global es claramente descendente.

La distancia recorrida también se reduce de forma notable. Penalizar la distancia provoca que Hill Climbing priorice rutas más cortas aún a costa de satisfacer menos peticiones y obtener menor beneficio.

En cuanto al tiempo de ejecución, no se observa un patrón claro.

Las últimas cuatro gráficas muestran cómo afecta el incremento del coste por km a la antigüedad de las peticiones satisfechas.

Observamos que las peticiones de ese mismo día son las más afectadas, pues su número cae con rapidez al aumentar el coste, pero se mantienen como las peticiones más frecuentes de entre los 4 grupos.

Las peticiones de hace 1, 2 y 3 días presentan oscilaciones, y es difícil observar una tendencia clara. Parece que están muy correlacionadas y cuando una de las tres disminuye las otras aumentan o se mantienen dejando un patrón individual poco claro.

Globalmente, todas las categorías experimentan una reducción en el número de peticiones satisfechas a medida que la penalización por distancia aumenta, pero las peticiones más recientes son las más aparentemente penalizadas; posiblemente porque constituyen una mayoría clara.

7. Influencia del horario de trabajo sobre el beneficio obtenido

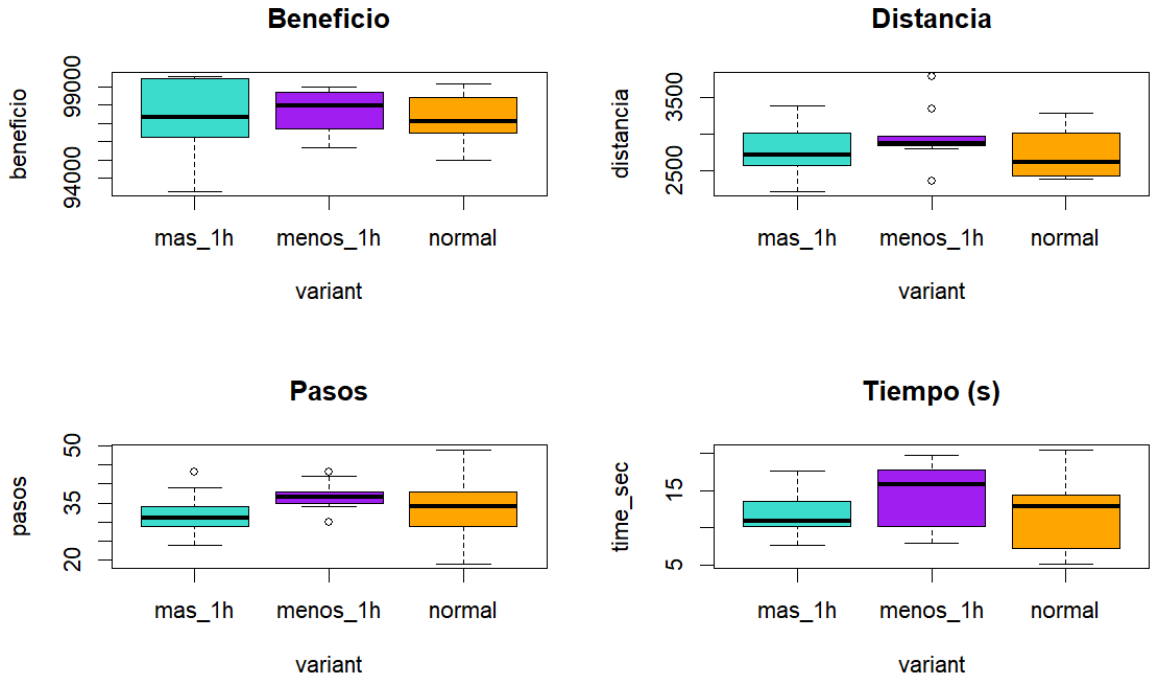
Para determinar la influencia del horario de trabajo sobre el beneficio obtenido se ha realizado el experimento 7.

Con este experimento se busca determinar cómo afecta disminuir o aumentar una hora el horario de trabajo de las cisternas. Se quiere ver si una hora más de trabajo puede aumentar el beneficio y una hora menos disminuirlo o si el algoritmo convergerá de forma similar dado el coste que implica recorrer cualquier distancia.

Para analizar este fenómeno se comparará los resultados obtenidos para el modelo con el número máximo de km por camión de 640 con dos modelos que difieren por 80 km, pues se ha indicado que los camiones van a 80 km/h implicando que una hora más, o menos, de servicio diario implican 80 km más, o menos, de trayecto.

Este experimento se hará a partir del estado inicial “greedy”, con los operadores “set_operadores_x” y con el algoritmo Hill Climbing.

La predicción de este experimento, es que dado este aumento diario de 640 km a 720 km, se debería permitir servir alguna petición adicional por camión, o completar más viajes, es decir, que hagan dos paradas que antes no podían por las restricciones por kilómetro. Esto provocará una incremento en el beneficio neto, al poder atender más peticiones. En cambio, si se reduce este límite a 560 km causaría el efecto contrario, haciendo que se formen más viajes unitarios, reduciendo las peticiones atendidas, lo que implica una reducción del beneficio por petición y un aumento de pérdidas potenciales.



Los resultados indican que variar el horario de trabajo una hora no tiene un impacto estadístico significativo. Observando los Boxplots no se identifica demasiado cambio de un modelo a otro.

Se ha decidido efectuar unos T-tests para verificar si realmente no está habiendo ningún cambio. Los test se han hecho con la hipótesis de que los modelos eran equivalentes y el único test en el que hemos rechazado dicha hipótesis ha sido el que comparaba la cantidad de pasos que ha tomado Hill Climbing para converger entre el modelo con 80 km más de recorrido por camión y el de 80 km menos.

Podemos concluir que variar una hora el horario de trabajo no mejora ni empeora el modelo significativamente.

Comparación entre Hill Climbing y Simulated Annealing

Finalmente, es de nuestro interés comparar el rendimiento de los dos algoritmos a la hora de resolver nuestro problema. A continuación, una tabla comparativa de los resultados Hill Climbing y Simulated Annealing, para el set de operadores, generación de estado inicial, heurística, y parámetros de *schedule* escogidos:

Algoritmo	Beneficio (€)	Coste* (€)	Beneficio neto (€)	Tiempo (s)
Hill Climbing	96734	5512.4	91222	10.89
Simulated Annealing	93200	6431.6	86768.4	0.88

*coste medido como km recorridos * 2

Como podemos observar, el beneficio neto de Hill Climbing es superior al de Simulated Annealing en estos experimentos concretos. Aunque Simulated Annealing obtiene soluciones con un coste mayor (y por tanto un beneficio neto menor), su principal ventaja radica en el tiempo de ejecución, que es más de diez veces menor que el de Hill Climbing. Esto sugiere que, para problemas donde el tiempo de cálculo es crítico o se requiere generar muchas soluciones en poco tiempo, Simulated Annealing puede ser preferible pese a producir resultados de menor calidad.